

1. หมวดเตรียมเกม (Game Setup)

init_game()

- หน้าที่: รีเซ็ตและตั้งค่าเริ่มต้นของระบบเกมทั้งหมดเพื่อเริ่มเล่นรอบใหม่
- Parameters: ไม่มี
- Return Value: ไม่มี (None)
- Global Variables ที่เปลี่ยนค่า: board, target_final_color, moves_left, game_state, selected_color
- รายละเอียดการทำงาน:
 - สร้างกระดาน 2 มิติขนาด 10 x 8 (GRID_COLS x GRID_ROWS) โดยการสุ่มเลือกสีจาก COLORS ใส่ในตาราง
 - สุ่มกำหนดสีเป้าหมายชนะ (target_final_color)
 - ตั้งค่าจำนวนครั้งที่เดินได้ (moves_left = 12)
 - เปลี่ยนสถานะเกมเป็น "PLAYING" และล้างค่าสีที่เลือก (selected_color = None)

2. หมวดกติกาและระบบเล่น (Game Rules & Logic)

spread(start_x, start_y, target_colour, new_colour)

- หน้าที่: เติส้แบบกระจายวงกว้าง (Flood Fill Algorithm) จากจุดเริ่มต้นไปยังช่องข้างเคียงที่มีสีเดียวกัน
- Parameters:
 - start_x (int) : พิกัด X บนกระดาน (0 ถึง 9)
 - start_y (int) : พิกัด Y บนกระดาน (0 ถึง 7)
 - target_colour (str) : สีเดิมตรงจุดเริ่มต้นที่ต้องการเปลี่ยน
 - new_colour (str) : สีใหม่ที่ต้องการแทนที่
- Return Value: ไม่มี (None)

check_win()

- หน้าที่: ตรวจสอบว่าผู้เล่นทำให้ทุกช่องบนกระดานเปลี่ยนเป็นสีเป้าหมาย (target_final_color) สำเร็จแล้วหรือยัง
- Parameters: ไม่มี
- Return Value: (bool)
 - True : หากทุกช่องบนกระดานมีสีตรงกับ target_final_color
 - False : หากมีอย่างน้อย 1 ช่องที่ไม่ตรงกับสีเป้าหมาย

change_colour(start_x, start_y, new_colour)

- หน้าที่: ประมวลผลเมื่อผู้เล่นคลิกเลือกสีลงบนกระดาน คำนวณการลดจำนวนการเดิน และเช็คเงื่อนไข ชนะ/แพ้
- Parameters:
 - start_x (int) : พิกัด X ของกระดานที่ถูกคลิก
 - start_y (int) : พิกัด Y ของกระดานที่ถูกคลิก
 - new_colour (str) : สีใหม่ที่ผู้เล่นเลือกเพื่อนำไปเท
- Return Value: ไม่มี (None)

3. กลุ่มฟังก์ชันแสดงผลกราฟิกและ UI (Graphics & Rendering)

`get_color_fill(colour)`

- หน้าที่: กำหนดสีพู่กัน (Fill color) ของ Processing ให้ตรงกับชื่อสีข้อความ
- Parameters:
 - `colour (str)` : ชื่อความสี ("RED", "GREEN", "BLUE", "YELLOW")
- Return Value: ไม่มี (None)

`draw_hud()`

- หน้าที่: วาดองค์ประกอบหน้าจอแสดงผลข้อมูล (Heads-Up Display) และข้อความจากจบเกม
- Parameters: ไม่มี
- Return Value: ไม่มี (None)

4. หมวดรับคำสั่งจากผู้เล่น (Player Input)

`mousePressed()`

- หน้าที่: ดักจับเหตุการณ์เมื่อผู้เล่นคลิกเมาส์ เพื่อเลือกสีหรือคลิกบนกระดาน
- Parameters: ไม่มี
- Return Value: ไม่มี (None)

SAVE GAME

1. โครงสร้างข้อมูลที่บันทึก

ข้อมูลสถานะที่จำเป็นต้องจัดเก็บลงไฟล์ `savegame.json`:

- `board`: ตารางสี 2 มิติ ขนาด 10×8
- `target_final_color`: สีเป้าหมายที่ต้องชนะ
- `moves_left`: จำนวนรอบที่เหลือ
- `game_state`: สถานะเกมปัจจุบัน (PLAYING, WON, LOST)
- `selected_color`: สีที่ผู้เล่นเลือกไว้ล่าสุด

2. Function Design

`save_game()`

- หน้าที่: รวบรวมสถานะปัจจุบันของเกม แปลงเป็นรายการข้อความ (List of Strings) แล้วเซฟลงไฟล์
- Input: ตัวแปรระบบ (`moves_left`, `target_final_color`, `game_state`, `board`)
- Output: ไฟล์ `savegame.txt` ในโฟลเดอร์ของโปรเจกต์
-

`load_game()`

- หน้าที่: อ่านไฟล์ `savegame.txt` แล้วดึงข้อมูลกลับมาแปลงค่าใส่ตัวแปรหลักของเกม
- Input: ไฟล์ `savegame.txt`
- Output: อัปเดตตัวแปร Global เพื่อเปลี่ยนสถานะกระดานและเกมบนหน้าจอ

`keyPressed()` (Event Controller)

- หน้าที่: ดักจับการกดคีย์บอร์ดจากผู้เล่น
- Input: ปุ่มคีย์บอร์ดที่ถูกกด (`key`)
- เช็คว่ากดปุ่ม 's' หรือ 'S' ---> เรียกฟังก์ชัน `save_game()`
- เช็คว่ากดปุ่ม 'l' หรือ 'L' ---> เรียกฟังก์ชัน `load_game()` แล้วสั่ง `redraw()` เพื่อรีเฟรชหน้าจอทันที